



Язык программирования C++

Лекция 3

ПСЕВДОНИМЫ И ОПРЕДЕЛЕНИЯ ТИПОВ

```
using identifier = type;
```

```
using counter = long;
```

```
using func = void(*) (int) ;
```

```
typedef type identifier;
```

```
typedef long counter;
```

```
typedef void (*func) (int) ;
```

ПЕРЕЧИСЛИМЫЕ ТИПЫ ДАННЫХ

```
enum [<type_name>] {<consts_list>;
```

```
enum Color
{
    color_red ,
    color_green ,
    color_blue ,
};

Color paint = color_green ;
Color house (color_blue );
Color apple { color_red };
```

```
enum Color
{
    color_red = 1,
    color_green ,
    color_blue ,
};

Color paint { color_green };
std::cout << paint;
```

ПРИМЕР

```
enum ParseResult {  
    success ,  
    error_opening_file ,  
    error_reading_file ,  
    error_parsing_file  
};
```

```
int readFileContents () {  
    if (!openFile ())  
        return -1;  
    if (!readFile ())  
        return -2;  
    if (!parseFile ())  
        return -3;  
    return 0;  
}
```

```
ParseResult readFileContents () {  
    if (!openFile ())  
        return error_opening_file ;  
    if (!readFile ())  
        return error_reading_file ;  
    if (!parsefile ())  
        return error_parsing_file ;  
    return success ;  
}
```

```
if (readFileContents () == success) {  
    // do smth  
}  
else {  
    // error message  
}
```

```
enum SortType {  
    sorttype_forward ,  
    sorttype_backwards  
};  
  
void sortData (SortType type) {  
    if (type == sorttype_forward )  
        // sort data in direct order  
    else if (type == sorttype_backwards )  
        // reverse order  
}
```

СТРУКТУРЫ

Структура – пользовательский тип данных, позволяющий группировать в себе набор переменных различных типов

- Элементы структуры называются полями структуры и могут иметь любой тип, кроме типа этой же структуры, но могут быть указателями на него

```
struct [name] { <member-list> } [declarators];
```

ПРИМЕРЫ

```
struct {  
    string name;  
    int code;  
    float wage;  
} staff[10], *ps;
```

```
typedef struct  
{  
    string name;  
    int code;  
    float wage;  
} Employee;
```

```
struct Employee {  
    string name;  
    int code;  
    float wage;  
};  
Employee staff[10], *ps;
```

ИНИЦИАЛИЗАЦИЯ СТРУКТУР

```
struct {  
    string name;  
    int code;  
    float wage;  
} employee = {"John", 123, 666.67};
```

```
staff[2]=staff[1];
```

```
Employee employee, staff[10], *ps;  
...  
employee.name = "John";  
staff[8].code = 123;  
ps->wage = 666.67;
```

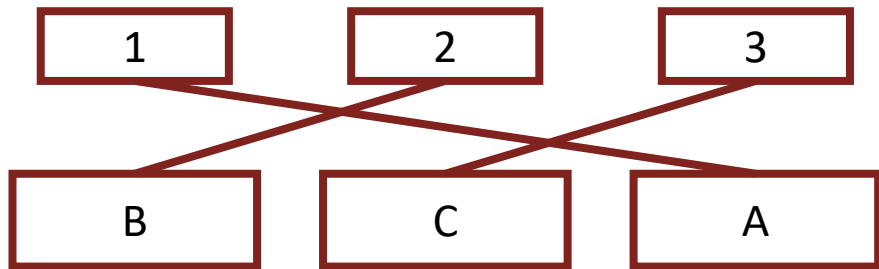
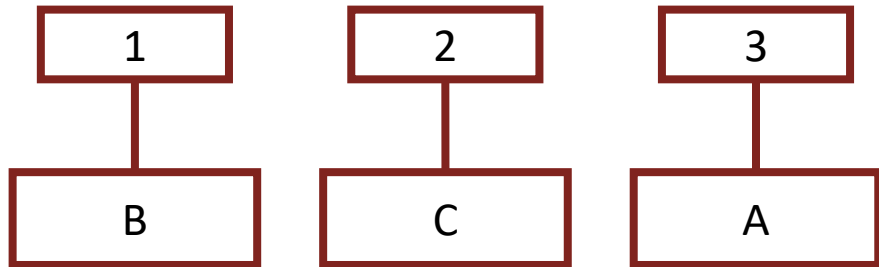
ВВОД И ВЫВОД СТРУКТУР

```
struct Students {  
    unsigned int num;  
    string fio;  
    unsigned short int kurs;  
};
```

```
for (int i = 0; i < 5; ++i) {  
    cout << stud[i].num << " " <<  
        stud[i].fio << " " <<  
        stud[i].kurs << '\n';  
}
```

```
Students stud[5];  
for (int i = 0; i < 5; i++) {  
    cout << "Enter number " << '\n';  
    cin >> stud[i].num;  
    cin.ignore();  
    cout << "Enter fio " << '\n';  
    getline (cin, stud[i].fio);  
    cout << "Enter kurs " << '\n';  
    cin >> stud[i].kurs;  
}
```

УКАЗАТЕЛИ НА СТРУКТУРЫ



```
#include <iostream>
#include <string>
using namespace std;
typedef struct
{
    string name;
    int code;
    float wage;
} Employee;

void Sort(Employee** p, int N) {
    for (int i = N - 1; i > 0; i--)
        for (int j = 0; j < i; j++)
            if (p[j]->name > p[j + 1]->name) {
                Employee* temp;
                temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
}

void Print(Employee** p, int N) {
    . . .
}

int main() {
    int N;
    cin >> N;
    Employee* staff = new Employee[N];
    . . .
    Employee** p = new Employee* [N];
    for (int i = 0; i < N; i++)
        p[i] = &staff[i];
    Sort(p, N);
    Print(p, N);
    return 0;
}
```

ОБЪЕДИНЕНИЯ

объединение может инициализироваться только значением его первого элемента

объединение не может содержать битовые поля

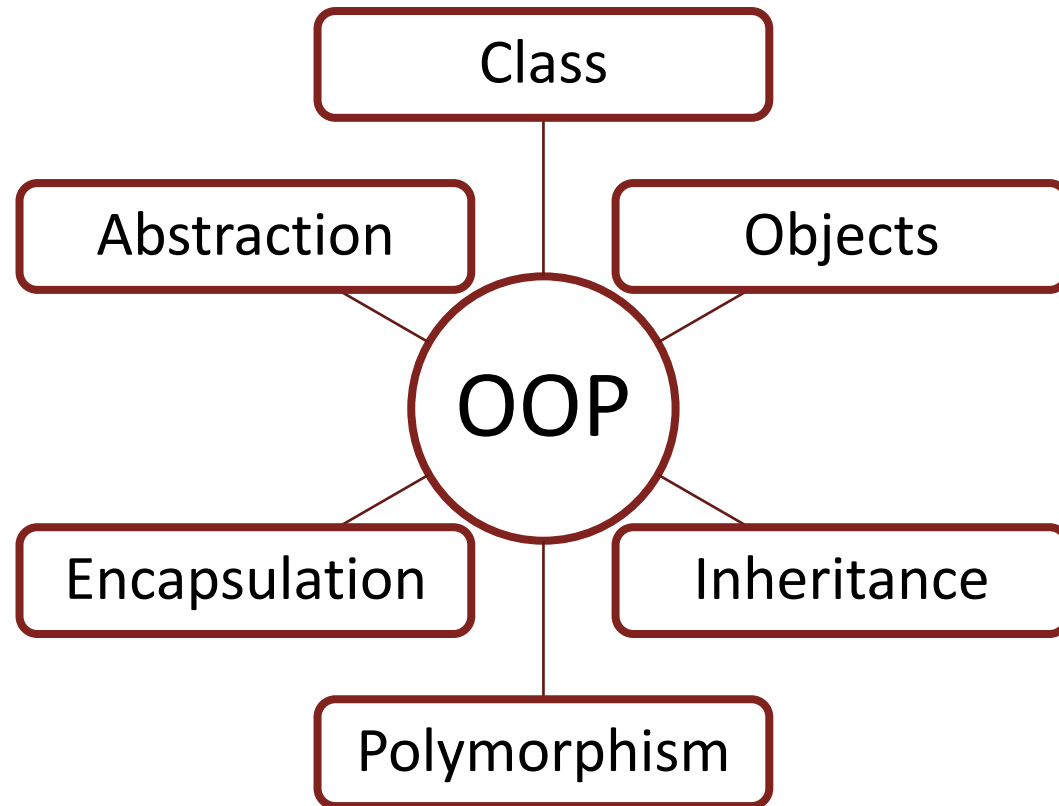
в качестве полей объединений можно использовать только базовые типы, а также структуры, объединения и классы

ПРИМЕР

```
int main() {
    enum paytype {CARD = 1, CHECK = 2};
    int ptype;
    union pay {
        char card[16];
        long summa;
    } info;
    cout<< "Способ оплаты";
    cin >> ptype;
    switch (ptype) {
        case CARD: cout << "Оплата по карте, введите номер карты: ";
            cin>>info.card; break;
        case CHECK: cout << "Оплата наличными ";
            cin>>info.summa; break;
    }
    return 0;
}
```

```
int main() {
    enum paytype {CARD = 1, CASH = 2};
    struct {
        int ptype;
        union pay {
            char card[16];
            long summa;
        } oplata;
    } info;
    cout<< "Способ оплаты";
    cin >> info.ptype;
    switch (info.ptype) {
        case CARD: cout << "Оплата по карте, введите номер карты:";
            cin>>info.oplata.card; break;
        case CASH: cout << "Оплата наличными, введите сумму";
            cin>>info.oplata.summa; break;
    }
    return 0;
}
```

OOP



КЛАССЫ И ЧЛЕНЫ КЛАССОВ

```
class Employee
{
public:
    std::string m_name;
    int m_id;
    double m_wage;
    // вывод информации о сотруднике
    void print();
};
```

```
// вывод информации о сотруднике
void Employee::print()
{
    std::cout << "Name: " << m_name <<
        " Id: " << m_id <<
        " Wage: " << m_wage << '\n';
}
```

```
int main()
{
    Employee jack{ "Jack", 1, 35.00 };
    Employee john{ "John", 2, 32.50 };

    jack.print();
    john.print();

    return 0;
}
```

СПЕЦИФИКАТОРЫ ДОСТУПА

```
class Employee
{
    string m_name{};
    int m_id{};
    double m_wage{};
};
```

```
Employee jack;
jack.m_name = "Jack";
```

public

private

protected

КОНТРОЛЬ ДОСТУПА НА ОСНОВЕ КЛАССА!!!

```
class DateClass
{
    int m_month;
    int m_day;
    int m_year;

public:
    void setDate(int month, int day, int year)
    {
        m_month = month;
        m_day = day;
        m_year = year;
    }

    void print() { ... }

    void copyFrom(const DateClass& d)
    {
        m_month = d.m_month;
        m_day = d.m_day;
        m_year = d.m_year;
    }
};
```

```
DateClass date;
date.m_day = 4;
date.setDate(10, 14, 2020);
DateClass copy;
copy.copyFrom(date);
```

```
✓ class Date
{
private:
    int m_month;
    int m_day;
    int m_year;

public:
    int getMonth() { return m_month; }
    void setMonth(int month) { m_month = month; }

    int getDay() { return m_day; }
    void setDay(int day) { m_day = day; }

    int getYear() { return m_year; }
    void setYear(int year) { m_year = year; }
};
```

ГЕТТЕРЫ И СЕТТЕРЫ

КОНСТРУКТОР

public:

Fraction()

{

m_numerator = 0;

m_denominator = 1;

}

Fraction(int numerator, int denominator = 1)

{

m_numerator = numerator;

m_denominator = denominator;

}

```
Fraction frac1;  
Fraction frac2(5, 8);  
Fraction frac3(5);
```

```
Fraction frac{13, 666};
```

```
Fraction(int numerator = 0, int denominator = 1)  
{  
    assert(denominator != 0);  
  
    m_numerator = numerator;  
    m_denominator = denominator;  
}
```

ДЕСТРУКТОР

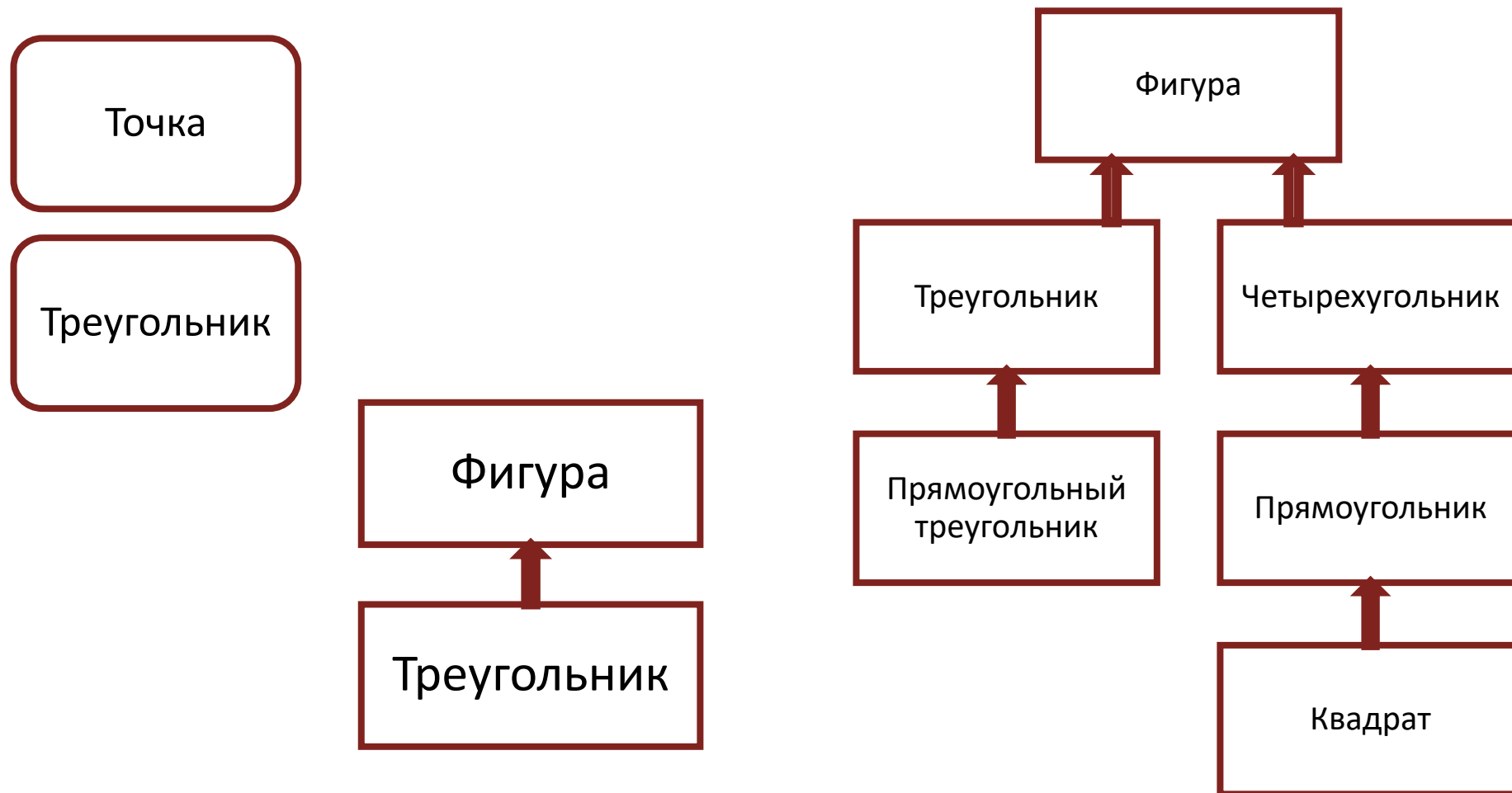
```
class IntArray
{
private:
    int* m_array{};
    int m_length{};

public:
    IntArray(int length)
    {
        assert(length > 0);

        m_array = new int[length] {};
        m_length = length;
    }
}
```

```
~IntArray()
{
    delete[] m_array;
}
```

КОМПОЗИЦИЯ, НАСЛЕДОВАНИЕ, ИЕРАРХИЯ



```
class Person
{
public:
    std::string m_name{};
    int m_age{};
    Person(const std::string& name = "", int age = 0)
        : m_name{ name }, m_age{ age }
    {
    }
    const std::string& getName() const { return m_name; }
    int getAge() const { return m_age; }
};
```

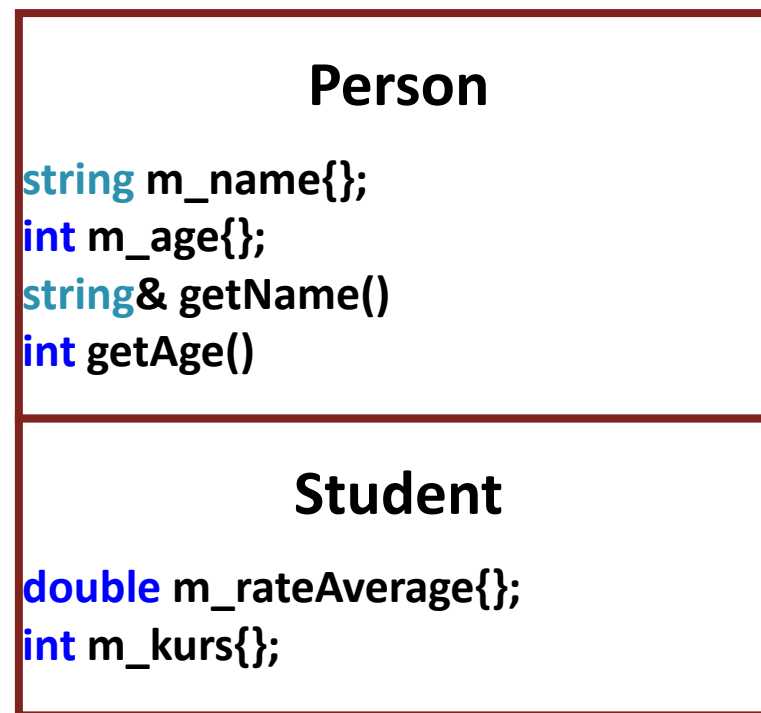
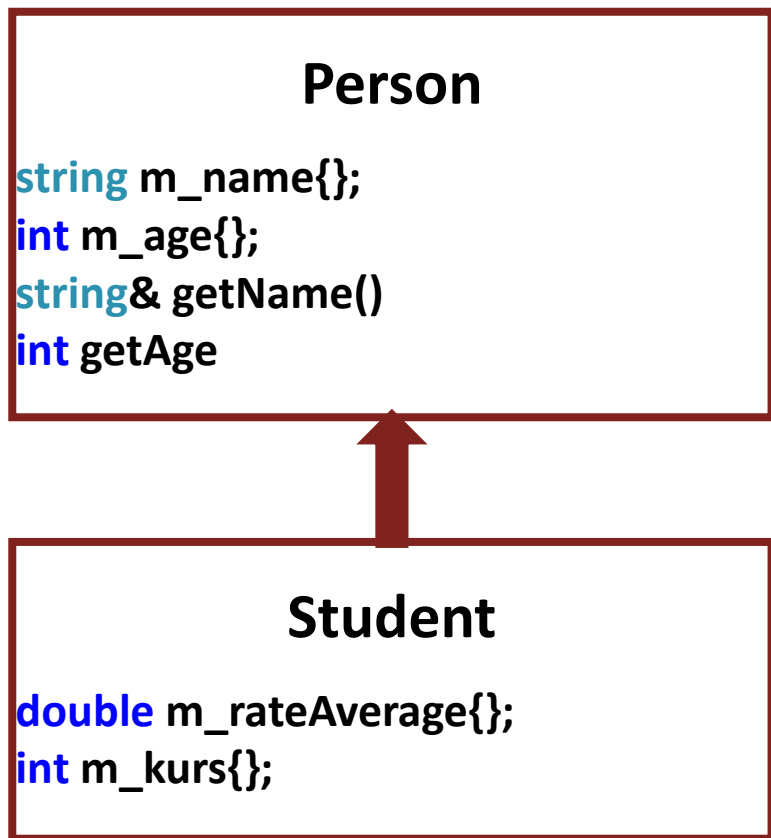
```
class Student: public Person
{
public:
    double m_rateAverage{};
    int m_kurs{};

    Student(double rateAverage = 0.0, int kurs = 0)
        : m_rateAverage{ rateAverage }, m_kurs{ kurs }
    {
    }
};
```

```
class Prepod : public Person
{
public:
    double m_hourlySalary{};
    long m_prepodID{};
    Prepod(double hourlySalary = 0.0, long prepodID = 0)
        : m_hourlySalary{ hourlySalary }, m_prepodID{ prepodID }
    {
    }
    void printNameAndSalary() const
    {
        std::cout << m_name << ": " << m_hourlySalary << '\n';
    }
};
```

```
class BigBoss : public Prepod
{
public:
    long m_preposIDs[100]{};
};
```

ПОРЯДОК СОЗДАНИЯ ОБЪЕКТОВ



```
class C1
{
public:
    C1()
    {
        std::cout << "C1\n";
    }
};

class C2 : public C1
{
public:
    C2()
    {
        std::cout << "C2\n";
    }
};

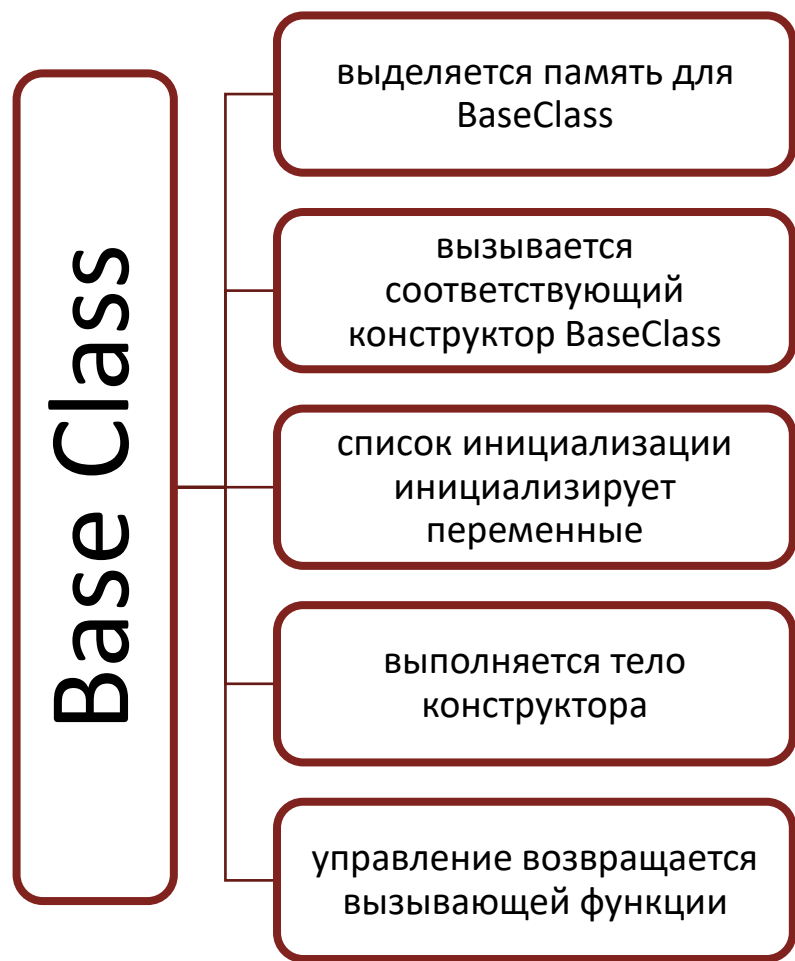
class C3 : public C2
{
public:
    C3()
    {
        std::cout << "C3\n";
    }
};

class C4 : public C3
{
public:
    C4()
    {
        std::cout << "C4\n";
    }
};
```

```
std::cout << "Constructing 1: \n";
C1 cC1;
std::cout << "Constructing 2: \n";
C2 cC2;
std::cout << "Constructing 3: \n";
C3 cC3;
std::cout << "Constructing 4: \n";
C4 cC4;
```

```
Constructing 1:
C1
Constructing 2:
C1
C2
Constructing 3:
C1
C2
C3
Constructing 4:
C1
C2
C3
C4
```

КОНСТРУКТОРЫ И ИНИЦИАЛИЗАЦИЯ



```
class BaseClass
{
private:
    int field_base;

public:
    BaseClass(int field_base = 0)
        : field_base{ field_base }
    {
    }

    int getFB() const { return field_base; }
};
```

```
class DerivedClass : public BaseClass
{
private:
    double field_derived;

public:
    DerivedClass(double field_derived = 0.0, int field_base = 0)
        : BaseClass{ field_base },
          field_derived{ field_derived }
    {
    }

    double getFD() const { return field_derived; }
};
```

СПЕЦИФИКАТОРЫ ДОСТУПА И НАСЛЕДОВАНИЕ

```
class BaseClass
{
public:
    int m_public;
protected:
    int m_protected;
private:
    int m_private;
};
```

```
class DerivedClass : public BaseClass
{
public:
    DerivedClass()
    {
        m_public = 1;
        m_protected = 2;
        m_private = 3;
    }
};
```

```
int main()
{
    BaseClass base_obj;
    base_obj.m_public = 1;
    base_obj.m_protected = 2;
    base_obj.m_private = 3;

    return 0;
}
```

ВИДЫ НАСЛЕДОВАНИЯ

Спецификатор доступа в базовом классе	Спецификатор доступа при открытом наследовании	Спецификатор доступа при закрытом наследовании	Спецификатор доступа при защищенном наследовании
public	public	private	protected
protected	protected	private	protected
private	inaccessible	inaccessible	inaccessible

Класс всегда может получить доступ к своим собственным (ненаследуемым) членам

Сторонний код получает доступ к членам класса на основе спецификаторов доступа класса, к которому он обращается

Производный класс обращается к унаследованным членам на основе спецификатора доступа, унаследованного от родительского класса. Это зависит от спецификатора доступа и типа наследования

